

**UNIVERSIDADE ESTADUAL PAULISTA “JÚLIO DE MESQUITA
FILHO”**
FACULDADE DE CIÊNCIAS
DEPARTAMENTO DE COMPUTAÇÃO
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

**RELATÓRIO PARCIAL DE TRABALHO DE CONCLUSÃO
DE CURSO**

Aluno: Kauã Junior Silva Soares
RA: 231024061
Orientador: Prof. Dr. Kelton Augusto Pontara da Costa
Título: Monitoramento de Ransomware em Nível de Kernel
Utilizando eBPF e Análise de Entropia
Período: Março a junho de 2026
Data: 22 de junho de 2026

1 RESUMO DO PROJETO

Foi proposto um agente de segurança em nível de kernel capaz de detectar e bloquear ataques de ransomware em tempo real. A ideia central vem de uma observação simples. Quando um ransomware cifra arquivos em massa, os dados que ele grava no disco têm entropia de Shannon perto do valor máximo (8 bits por byte), porque o conteúdo cifrado fica parecido com uma sequência aleatória. Esse comportamento não depende do algoritmo de cifragem usado e é difícil de mascarar entre variantes do malware.

O agente é escrito em C com a biblioteca libbpf e roda sobre o eBPF (extended Berkeley Packet Filter) do kernel Linux. Programas eBPF são executados dentro do kernel, em ambiente isolado (sandbox), e não precisam de instalação de módulos, o que mantém os experimentos seguros e controlados. A atividade de escrita é observada por kprobes, que se conectam às funções de escrita do sistema de arquivos. Os dados coletados por processo saem do kernel e vão para um componente em espaço de usuário, que calcula a entropia e decide se bloqueia o processo.

Os resultados serão comparados com ferramentas que já existem, como o auditd e detectores baseados em inotify, medindo o tempo de detecção, a taxa de acerto e o impacto no desempenho do sistema. O ambiente de desenvolvimento e de testes usa kernel Linux 5.15 ou superior em arquitetura x64, com suporte a BTF e ao modelo de compilação CO-RE (Compile Once, Run Everywhere).

2 ATIVIDADES DESENVOLVIDAS NO PERÍODO

2.1 Revisão Bibliográfica (março a junho, contínua)

A revisão bibliográfica começou em março e continua em andamento, acompanhando o que o projeto vai precisando. Comecei estudando a teoria de detecção de ransomware baseada em comportamento. O trabalho de Continella et al. (2016) sobre o ShieldFS me ajudou a embasar a ideia de monitorar a atividade no sistema de arquivos. Scaife et al. (2016) seguem a mesma linha com o CryptoDrop, que detecta ransomware olhando as alterações feitas nos arquivos do usuário. Já o trabalho de Davies et al. (2020) trouxe uma crítica importante, mostrando as limitações de confiar só na entropia (como bloquear por engano arquivos que já são comprimidos).

O artigo de Fukuda et al. (2023) mostra na prática como defender sistemas contra ransomware em tempo real, juntando módulos de kernel, machine learning e análise de entropia.

Na parte de infraestrutura, usei o livro de Gregg (2019) sobre ferramentas de observação com BPF, a documentação oficial do kernel Linux e o guia de Nakryiko (2020) sobre o modelo CO-RE. Esse material orientou o uso do CO-RE, que deixa o mesmo código rodar em versões diferentes do kernel sem precisar recompilar. É essa base teórica que sustenta o trabalho até aqui.

2.2 Caracterização do Comportamento de E/S de Ransomware (abril a maio)

Nesta etapa analisei os padrões de escrita em disco gerados por processos de cifragem em massa, olhando como os valores dos bytes se distribuem. Cargas de trabalho normais, como edição de texto, acesso a banco de dados e manipulação de imagens não comprimidas, geram uma distribuição de bytes bem desigual. Já a cifragem com algoritmos modernos produz uma saída perto de uma distribuição uniforme, com entropia de Shannon que costuma passar de 7,9 bits por byte.

Isso confirmou que a entropia serve como indicador de detecção, mas também deixou claro um ponto de ambiguidade. Arquivos já comprimidos ou codificados (.gz, .zip, .jpg, .mp4) também geram escritas de entropia alta. Por causa disso passei a adotar um segundo critério junto

com a entropia, que é o volume total de dados escritos e a dispersão dessas escritas por vários arquivos. Esse padrão é típico de ransomware e pouco comum em programas de compressão ou conversão.

2.3 Projeto da Arquitetura do Agente eBPF (maio a junho)

Com base na bibliografia e na caracterização do comportamento, a arquitetura do agente foi definida e já está fechada. As decisões principais de projeto estão descritas abaixo.

O ponto de interceptação escolhido é um kprobe na função `vfs_write`, da camada VFS do kernel. Essa função é o caminho comum por onde passam as escritas no sistema de arquivos, não importa qual chamada de sistema o processo use (`write`, `pwrite` e outras). Por isso ela dá uma cobertura mais ampla do que ficar monitorando chamadas de sistema individuais por tracepoints.

As estatísticas de escrita ficam guardadas por processo num mapa do tipo `BPF_MAP_TYPE_HASH`, indexado pelo identificador do grupo de processos. Cada entrada guarda a contagem de frequência dos 256 valores possíveis de um byte, calculada sobre amostras de tamanho fixo lidas a cada escrita. Esse tamanho fixo não é escolha de projeto e sim uma exigência do verificador eBPF, que só aceita leitura da memória do espaço de usuário pelo helper `bpf_probe_read_user` quando o tamanho é conhecido em tempo de compilação.

O cálculo da entropia de Shannon é feito no espaço de usuário, porque o verificador eBPF não permite operação de ponto flutuante dentro do kernel. O componente em espaço de usuário recebe os dados acumulados por processo e aplica a fórmula $H = - \sum_{i=0}^{255} p_i \cdot \log_2 p_i$, em que p_i é a frequência relativa do byte i sobre o total de bytes amostrados. Os alertas saem do kernel para o espaço de usuário por um ring buffer (`BPF_MAP_TYPE_RINGBUF`).

A decisão de bloqueio depende de três condições ao mesmo tempo. A entropia precisa estar acima de um limiar (valor inicial de 7,8, ainda a calibrar nos experimentos), o volume total de dados escritos precisa passar de um mínimo (64 KB como valor de partida) e as escritas precisam estar espalhadas por um número mínimo de arquivos. Quando as três batem, o agente mata o processo suspeito mandando o sinal `SIGKILL`.

2.4 Implementação do Componente de Kernel (maio a junho, em andamento)

A implementação do componente eBPF está em andamento. As estruturas de dados compartilhadas entre o programa de kernel e o espaço de usuário já estão definidas num arquivo comum (`common.h`), que são a estrutura com as estatísticas de cada processo e a estrutura do alerta mandado pelo ring buffer.

O ambiente de desenvolvimento foi montado com clang 18, llvm 18, libbpf 1.3 e bpftool 7.4, num Ubuntu com kernel 5.15 e suporte a BTF. O arquivo vmlinux.h, que o CO-RE precisa, foi gerado com o bpftool a partir dos metadados BTF do próprio kernel em execução.

O programa eBPF base, responsável por interceptar `vfs_write`, ler a amostra, somar o histograma no mapa e emitir eventos para o ring buffer, foi escrito levando em conta as restrições do verificador. Entre os cuidados estão o `#pragma unroll` em todos os laços que percorrem a amostra, uma máscara explícita no índice do histograma (para o verificador conseguir provar que ele fica no intervalo de 0 a 255) e o padrão de consultar o mapa antes de atualizar cada entrada. O Makefile do projeto cuida de compilar o programa eBPF, gerar o skeleton com o bpftool e compilar o componente de espaço de usuário.

A verificação funcional, que é confirmar que o programa carrega, passa pelo verificador do kernel e gera eventos no ring buffer diante de escritas reais, está em andamento e é a prioridade das próximas semanas.

2.5 Redação da Monografia (abril a junho, em andamento)

A redação da monografia começou em abril. Os capítulos em andamento são a introdução, o referencial teórico e a metodologia. O referencial cobre a entropia de Shannon aplicada à análise de dados cifrados, o funcionamento do eBPF no kernel Linux, os recursos do CO-RE e as restrições do verificador. A metodologia descreve o ambiente de testes, os critérios de detecção e o protocolo de experimentos com amostras de ransomware em ambiente isolado.

3 CRONOGRAMA ATUALIZADO

A Tabela 1 mostra o cronograma atualizado em relação ao que foi planejado na proposta, com a situação de cada atividade na data deste relatório (22 de junho de 2026).

4 ATIVIDADES PENDENTES

As atividades pendentes para o segundo semestre de 2026 seguem o cronograma original, sem precisar replanejar. As prioridades de cada período estão descritas abaixo.

Em julho a prioridade é fechar a implementação do componente de kernel, com a verificação funcional confirmando a interceptação das escritas, a soma correta do histograma por processo e a emissão de eventos pelo ring buffer. Em paralelo, fechar o componente de espaço de usuário, que cuida do cálculo de entropia, da exibição dos alertas e da política de bloqueio.

Tabela 1: Cronograma atualizado, situação em 22/06/2026

Atividade	Mar	Abr	Mai	Jun	Jul	Ago	Set	Out	Situação
Revisão bibliográfica					○	○	○	○	Contínua
Análise de comportamento									Concluída
Projeto da arquitetura eBPF									Concluído
Implementação (kernel)					○				Em andamento
Implementação (espaço usuário)					○				Iniciada
Validação funcional					○	○			Pendente
Benchmark e comparativo						○	○		Pendente
Redação da monografia					○	○	○	○	Em andamento
Ajustes finais								○	Pendente
Concluído ○ Planejado/em andamento									

Até o fim do mês o sistema já deve detectar e bloquear o simulador de cifragem em massa no ambiente de desenvolvimento.

Entre julho e agosto vem a validação do agente contra amostras reais de ransomware, numa máquina virtual isolada e sem acesso à rede. O critério é o bloqueio acontecer antes de o diretório alvo ser cifrado por inteiro, como está nos objetivos específicos da proposta.

Entre agosto e setembro entram as medições de desempenho com a ferramenta fio, comparando a eficiência e a latência de E/S com e sem o agente ativo, em cargas de trabalho diferentes. Depois vem o comparativo do agente com o auditd e com detectores baseados em inotify, avaliando o atraso de detecção, os arquivos perdidos antes do bloqueio, a taxa de verdadeiros positivos e o impacto no sistema.

A redação e a revisão da monografia seguem o tempo todo, de abril a outubro. Os capítulos de resultados vão sendo escritos junto com os experimentos.

Em outubro ficam os ajustes finais, a revisão completa com o orientador e a preparação da apresentação para a banca.

REFERÊNCIAS

CONTINELLA, Andrea et al. ShieldFS: a self-healing, ransomware-aware filesystem. In: ANNUAL COMPUTER SECURITY APPLICATIONS CONFERENCE, 32., 2016. *Anais* [...]. New York: ACM, 2016. p. 336–347.

DAVIES, [completar autores]. [completar título]. [completar veículo ou editora], 2020. [COMPLETAR REFERÊNCIA]

FUKUDA, [completar autores]. [completar título]. [completar veículo ou editora], 2023. [COMPLETAR REFERÊNCIA]

GREGG, Brendan. *BPF performance tools: linux system and application observability*. Boston: Addison-Wesley, 2019.

LINUX KERNEL DOCUMENTATION. BPF subsystem. Disponível em: <https://www.kernel.org/doc/html/latest/bpf/index.html>. Acesso em: mar. 2026.

NAKRYIKO, Andrii. BPF CO-RE (compile once, run everywhere). 2020. Disponível em: <https://nakryiko.com/posts/bpf-core-reference-guide/>. Acesso em: mar. 2026.

SCAIFE, Nolen et al. CryptoLock (and drop it): stopping ransomware attacks on user data. In: IEEE INTERNATIONAL CONFERENCE ON DISTRIBUTED COMPUTING SYSTEMS, 36., 2016. *Anais [...]*. IEEE, 2016. p. 303–312.